

Aula 15 - Dominando Joins e Agregações em SQL

Descrição: Capacitar os alunos a entender e construir consultas complexas cruzando múltiplas tabelas, lidando com dados incompletos (nulos) e extraindo métricas sumarizadas usando a ordem correta de execução do SQL.

1. O Universo dos Joins (Junções)

Neste primeiro momento, o foco é entender **como** o banco de dados conecta informações espalhadas. Antes de qualquer código, você precisa entender a arquitetura.

1.1 Fundamentos de Relacionamento

As junções cruzam informações baseadas em **Chaves Primárias (PK)** e **Chaves Estrangeiras (FK)**. Na modelagem, dividimos os dados em:

- **Tabela Dimensão:** Contém características estáticas (ex: nome do departamento, dados do cliente), em que as chaves primárias não se repetem.
- **Tabela Fato:** Registra eventos (ex: vendas, realizações de cursos), e utiliza chaves estrangeiras para referenciar as dimensões.

1.2 O Produto Cartesiano e a Condição de Junção

Para explicar o **Produto Cartesiano** e a **Condição de Junção**, vamos usar uma analogia do dia a dia: o cruzamento de peças de roupa.

O que é o Produto Cartesiano? (A "Explosão" de Dados)

Imagine que você tem duas listas (tabelas):

- **Tabela A (Cores de Camisas):** [Azul, Branco]
- **Tabela B (Tipos de Calças):** [Jeans, Moletom, Social]

Se você pedir ao SQL para unir essas tabelas sem dar nenhuma instrução de como elas se combinam (`SELECT * FROM Camisas, Calças`), o banco de dados fará o **Produto Cartesiano**. Ele vai criar **todas as combinações possíveis**, mesmo as que não fazem sentido.

O resultado seria:

1. Azul com Jeans
2. Azul com Moletom
3. Azul com Social
4. Branco com Jeans
5. Branco com Moletom
6. Branco com Social

Por que isso é perigoso? Se você tem uma tabela de 1.000 instrutores e outra de 1.000 cursos, o produto cartesiano geraria **1.000.000 de linhas** (1.000 x 1.000). A maioria dessas linhas diria que o "Professor de Matemática ministra o curso de Culinária", o que é um dado falso e inútil.

A Condição de Junção (O "Filtro Inteligente")

Para que essa "explosão" de dados se torne uma informação útil, precisamos de um critério de correspondência. É aqui que entram as **Chaves Primárias (PK)** e **Estrangeiras (FK)**.

Voltando ao exemplo (`instrutor` e `ministra`):

- Cada instrutor tem um **ID** único.
- A tabela `ministra` também tem uma coluna **ID**, indicando quem deu aquela aula.

A **Condição de Junção** diz ao SQL: *"Só me mostre as linhas onde o ID do instrutor na Lista A seja exatamente igual ao ID que aparece na Lista B"*.

Na prática (Sintaxe SQL):

Forma Antiga (usando WHERE):

SQL

```
SELECT nome, id_curso
FROM instrutor, ministra
WHERE instrutor.ID = ministra.ID;
```

Aqui, o SQL primeiro "imagina" o produto cartesiano gigante e depois o **WHERE** atua como uma tesoura, cortando todas as combinações onde os IDs não batem.

Forma Moderna (usando JOIN ... ON):

SQL

```
SELECT nome, id_curso
FROM instrutor
JOIN ministra ON instrutor.ID = ministra.ID;
```

Esta forma é mais legível e eficiente, pois deixa claro que a união só existe por causa daquela condição específica.

1.3 Tipos de Junções

Existem diferentes comportamentos ao juntarmos as tabelas:

- **INNER JOIN (ou apenas JOIN):** O "filtro rigoroso" que retorna apenas as tuplas onde há correspondência exata em ambas as tabelas.
- **LEFT (OUTER) JOIN:** Retorna todas as linhas da tabela à esquerda, mesmo que não haja correspondência na direita (preenchendo esses vazios com **NULL**).
- **RIGHT (OUTER) JOIN:** Oposto do Left Join; prioriza e preserva todas as tuplas da relação à direita.
- **FULL (OUTER) JOIN:** Combina o Left e o Right; traz tudo de ambas as tabelas, inserindo nulos onde não houver o par.

1.4 Sintaxe de Junção: ON, USING e NATURAL JOIN

- **ON:** A condição **ON** define um predicado de junção explícito de forma muito parecida com o **WHERE**, permitindo junções complexas (ex: **ON aluno.ID = realiza.ID**).
- **USING:** Forma simplificada utilizada apenas quando os atributos a serem cruzados possuem exatamente o mesmo nome em ambas as tabelas.
- **NATURAL JOIN:** Une automaticamente as tabelas utilizando **todos** os atributos que tenham nomes idênticos.

Cuidado: se tabelas tiverem atributos em comum que não devem ser a chave da relação (ex: uma coluna chamada **nome** na tabela aluno e **nome** na tabela departamento), o natural join fará cruzamentos indesejados e incorretos.

1.5 EXEMPLO DE CÓDIGO

Para ilustrar os conceitos de **Joins** e **Relacionamentos**, vamos criar um cenário prático baseado nas fontes analisadas. Imagine um sistema de uma universidade com duas tabelas simples: uma de **Instrutores** (Dimensão) e outra de **Cursos** que eles ministram (Fato).

SQL

-- 1. Tabela Dimensão com coluna de salário

```
CREATE TABLE instrutor (
  ID INT PRIMARY KEY,
  nome VARCHAR(50),
  nome_dept VARCHAR(50),
```

```
salario DECIMAL(10,2) -- Alterado para DECIMAL para permitir cálculos
);
```

-- 2. Tabela Fato

```
CREATE TABLE ministra (
  ID_instrutor INT,
  id_curso VARCHAR(10),
  FOREIGN KEY (ID_instrutor) REFERENCES instrutor(ID)
);
```

```
INSERT INTO instrutor VALUES (1, 'Srinivasan', 'Comp. Sci.', 65000);
INSERT INTO instrutor VALUES (2, 'Wu', 'Finance', 90000);
INSERT INTO instrutor VALUES (3, 'Mozart', 'Music', NULL);
```

```
INSERT INTO ministra VALUES (1, 'CS-101'), (1, 'CS-315'), (2, 'FIN-201');
```

EXPLICAÇÃO DETALHADA

1. Fundamentos: PK e FK

No código acima, o **ID** na tabela **instrutor** é a **Chave Primária (PK)**. Ele é único. Na tabela **ministra**, o **ID_instrutor** é a **Chave Estrangeira (FK)**. Ela serve como um "apontador" que diz: "este curso foi dado pelo instrutor que possui esse ID na outra tabela".

2. Produto Cartesiano vs. Condição de Junção

Se você executar apenas `SELECT * FROM instrutor, ministra;`, o SQL fará o **Produto Cartesiano**.

- Como temos 3 instrutores e 3 registros de cursos, o resultado teria **9 linhas** (3x3).
- Veríamos combinações absurdas, como o instrutor de Finanças dando aula de Computação.

Para corrigir isso, usamos a **Condição de Junção**:

SQL

```
SELECT nome, id_curso
FROM instrutor
JOIN ministra ON instrutor.ID = ministra.ID_instrutor;
```

O **ON** atua como o "filtro inteligente", garantindo que o banco de dados só exiba a linha se o ID coincidir em ambos os lados.

3. Tipos de Junções (A diferença no resultado)

Baseado nos dados que inserimos, veja como o resultado muda conforme o tipo de Join:

- **INNER JOIN**: Retornaria apenas Srinivasan e Wu. **Mozart ficaria de fora**, pois ele não tem correspondência na tabela **ministra** (não ministra cursos).
- **LEFT JOIN**: Retornaria Srinivasan, Wu e **Mozart**. Como Mozart não tem curso, a coluna **id_curso** apareceria como **NULL** para ele. É "leal" à tabela da esquerda (**instrutor**).
- **FULL JOIN**: Traria todos os instrutores e todos os cursos, mesmo aqueles (se houvesse) sem nenhum instrutor vinculado.

4. Sintaxe: ON, USING e NATURAL JOIN

- **ON**: É o mais flexível. Usamos `ON instrutor.ID = ministra.ID_instrutor` porque os nomes das colunas são diferentes.
- **USING**: Se ambas as colunas se chamassem **ID**, poderíamos usar `JOIN ministra USING (ID)`.

- **NATURAL JOIN:** O SQL tentaria adivinhar a conexão. Se as duas tabelas tivessem uma coluna chamada `nome` (uma para o nome do instrutor e outra para o nome do curso), o **NATURAL JOIN** tentaria cruzar os nomes, o que daria erro na lógica da consulta, pois nomes de pessoas e cursos não costumam ser iguais.
-

2. O Desafio dos Valores Nulos (**NULL**)

Para ilustrar o **Desafio dos Valores Nulos (NULL)**, vamos expandir o nosso cenário de universidade. O segredo aqui é entender que o **NULL** não é o número zero nem um texto vazio; ele é um "buraco" na informação.

Exemplo de Código SQL: O Caso dos Instrutores sem Salário

SQL

-- 1. Inserindo dados com valores NULL

-- O instrutor 'Gold' acabou de ser contratado e seu salário é desconhecido (NULL).

INSERT INTO instrutor (ID, nome, nome_dept, salario)

VALUES (4, 'Gold', 'Physics', **NULL**);

-- 2. O ERRO COMUM: Tentar usar o sinal de igual (=)

-- Esta consulta NÃO retornará nada, pois '**NULL** = **NULL**' resulta em 'Desconhecido'.

SELECT nome **FROM** instrutor **WHERE** salario = **NULL**;

-- 3. A FORMA CORRETA: Usando predicados especiais

-- Para encontrar quem não tem salário definido:

SELECT nome

FROM instrutor

where salario **IS NULL**;

-- Para encontrar todos os que JÁ possuem salário definido:

SELECT nome

FROM instrutor

WHERE salario **IS NOT NULL**;

-- 4. NULL no DISTINCT (Comportamento de Identidade)

-- Se houver dois professores com salário NULL, o **DISTINCT** os verá como "iguais".

SELECT DISTINCT salario **FROM** instrutor;

Explicação Detalhada do Conceito

1. A Lógica do "Desconhecido" (Unknown)

Diferente de um número comum, o SQL usa uma **lógica ternária**. Em vez de apenas *Verdadeiro* ou *Falso*, existe o *Desconhecido*.

- Se você perguntar: "O salário do Gold (NULL) é igual a 5000?", o SQL responde: "Não sei".
- Se você perguntar: "O salário do Gold (NULL) é diferente de 5000?", o SQL responde: "Não sei".
- Por isso, operadores como `=` ou `<>` falham com nulos.

2. Predicados **IS NULL** e **IS NOT NULL**

Como o sinal de igual não funciona, o SQL fornece "ferramentas de teste" específicas. O **IS NULL** é a única forma confiável de verificar se um campo está vazio ou se a informação é não aplicável.

3. O Paradoxo do **DISTINCT** e Operações de Conjunto

Aqui o SQL abre uma exceção para facilitar a vida do usuário:

- **Nos Filtros (WHERE):** NULL nunca é igual a NULL.
- **No Agrupamento (DISTINCT, UNION, GROUP BY):** O SQL trata todos os NULLs como se fossem a mesma coisa para não repetir linhas "vazias" várias vezes no resultado final.

3. Resumindo com Funções de Agregação

Dando sequência ao seu aprendizado, agora que já resolvemos os erros de inserção e entendemos como o SQL lida com espaços vazios (NULL), vamos entrar na fase de inteligência: **como resumir milhares de dados em uma única resposta útil.**

Imagine que o diretor da faculdade não quer ver a lista de todos os 5.000 professores (as tuplas individuais). Ele quer apenas um número: "Quanto gastamos por mês?". A **Agregação** faz exatamente isso: ela "espreme" uma coleção de várias linhas até que saia um único valor resumido.

3.1 As 5 Funções Principais

Utilizando os dados da nossa tabela `instrutor` (Srinivasan, Wu, Mozart e Gold), veja como as funções se comportam:

A. **COUNT()** (Contagem):

- `COUNT(*)`: Conta todas as linhas. Se você rodar no seu script, dará **4**.
- `COUNT(salario)`: Olha apenas para a coluna salário. Como Mozart e Gold têm NULL, o SQL os ignora e retorna **2**.

B. **SUM()** (Soma):

Soma os valores numéricos. No nosso caso, somará $65.000 + 90.000 = \mathbf{155.000}$. Os nulos são ignorados.

C. **AVG()** (Média):

O SQL é inteligente. Ele soma os valores e divide **apenas** pela quantidade de registros não nulos. $(155.000 / 2 = \mathbf{77.500})$.

D. **MAX()** e **MIN()** (Extremos):

- `MAX(salario)` retornaria 90.000.
- Também funciona com textos! `MIN(nome)` retornaria "Gold" (primeiro na ordem alfabética).

3.2 DISTINCT em Agregações

Às vezes, você não quer saber quantos salários existem, mas sim quantas **faixas salariais diferentes** a empresa possui.

- Se tivermos dois professores ganhando exatamente 65.000, o `COUNT(salario)` diria "2".
- O `COUNT(DISTINCT salario)` diria "**1**", pois ele elimina as duplicatas antes de contar.

SQL

-- Exemplo: Quantos departamentos diferentes têm professores cadastrados?

```
SELECT COUNT(DISTINCT nome_dept) FROM instrutor;
```

3.3 A Armadilha do Agrupamento vs. Atributos Simples

Este é o erro que mais frustra iniciantes. Tente mentalizar esta consulta:

"Quem é o professor com o maior salário?"

Muitos tentam escrever:

```
SELECT nome, MAX(salario) FROM instrutor; (ISSO GERA ERRO!)
```

Por que o SQL trava?

Pense logicamente: o `MAX(salario)` resulta em **uma única linha** (o valor 90.000). Mas a coluna `nome` tem **quatro linhas** diferentes. O banco de dados não sabe qual nome "colar" ao lado do valor máximo.

As duas soluções permitidas:

1. **Usar o GROUP BY:** Você diz ao SQL para criar pequenos grupos e aplicar a função em cada um.

SQL

-- "Para cada departamento, me dê o maior salário"

SELECT nome_dept, MAX(salario)

FROM instrutor

GROUP BY nome_dept;

2. **Usar Subconsultas Aninhadas (Para o caso do "Quem é"):** Se você quer o nome da pessoa específica, você faz uma pergunta dentro da outra.

SQL

-- "Selecione o nome de quem tem o salário igual ao maior salário de todos"

SELECT nome

FROM instrutor

WHERE salario = (SELECT MAX(salario) FROM instrutor);

Resumo Didático

As funções de agregação são como **filtros de café**: você joga uma grande quantidade de pó (dados brutos) e, ao final, só sai a essência (o resultado calculado). O segredo é nunca tentar misturar o "pó" (nomes individuais) com a "essência" (o total), a menos que você use o **GROUP BY** para organizar a mistura.

4. Agrupamentos e Filtros (GROUP BY e HAVING)

Dando continuidade na aula, já sabemos como somar e tirar médias, vamos aprender a organizar esses cálculos de forma inteligente. No SQL, isso é feito com o **GROUP BY** (o organizador) e o **HAVING** (o selecionador de elite).

Até agora, as nossas funções de agregação (como **SUM**) olhavam para a tabela inteira e davam um resultado único. Mas, na vida real, precisamos de **detalhamento**. Em vez de "Qual a soma de todos os salários?", queremos saber "Qual a soma dos salários **por departamento**?".

4.1 O Papel do GROUP BY

O **GROUP BY** funciona como uma separação por "balaios". Ele diz ao SQL: "Antes de somar, separe as linhas em grupos que tenham o mesmo valor nesta coluna".

Exemplo Prático com nosso código:

SQL

-- Queremos a soma dos salários por departamento

SELECT nome_dept, SUM(salario)

FROM instrutor

GROUP BY nome_dept;

Aqui, o SQL cria um grupo para 'Comp. Sci.', outro para 'Finance', etc., e calcula a soma separadamente para cada um.

4.2 A Regra de Ouro do Rigor Técnico

Este é o ponto onde a maioria dos estudantes comete erros. Imagine que você escreve:

SELECT nome, nome_dept, SUM(salario) FROM instrutor GROUP BY nome_dept;

Isso dará erro! Porquê?

- O SQL criou um único grupo para 'Comp. Sci.'.
- Dentro desse grupo, existem vários nomes (ex: Srinivasan, etc.).
- O SQL não sabe qual nome mostrar na linha que resume o grupo.

A regra é clara: Se uma coluna está no `SELECT` e **não** tem uma função (como `SUM` ou `AVG`) em volta dela, ela **tem obrigatoriamente** que estar na lista do `GROUP BY`.

4.3 HAVING: O Filtro de Grupos

Muitos alunos confundem o `WHERE` com o `HAVING`. A diferença é o **momento** em que eles agem:

1. **WHERE (O porteiro):** Age **antes** de tudo. Ele barra as linhas individuais. Ex: "Não conte professores com salário menor que 1000".
2. **HAVING (O auditor):** Age **depois** que o grupo foi formado e a conta foi feita. Ele filtra o resultado da agregação.

Exemplo Comparativo:

SQL

`SELECT` nome_dept, `AVG`(salario)

`FROM` instrutor

`WHERE` nome_dept <> 'Music' -- 1. Primeiro, remove o departamento de música

`GROUP BY` nome_dept -- 2. Agrupa os que sobraram

`HAVING` `AVG`(salario) > 70000; -- 3. Só mostra os grupos cuja MÉDIA final foi > 70k

Dica Didática: O `WHERE` não conhece o resultado da soma. Se você tentar usar `WHERE SUM(salario) > 1000`, o SQL vai reclamar porque, no passo do `WHERE`, a soma ainda nem foi calculada! Para filtrar somas, médias ou contagens, use sempre o **HAVING**.

5. Ordem Lógica e Execução (O Funil do SQL)

Para encerrar nossa aula com chave de ouro, vamos entender o "cérebro" do banco de dados. Um dos maiores erros de quem aprende SQL é tentar ler o código da esquerda para a direita, como se fosse um livro. No entanto, o SQL é uma linguagem **declarativa**: você diz o que quer, e o banco decide como buscar. Para isso, ele segue uma ordem lógica interna que é bem diferente da ordem que escrevemos.

Imagine que o banco de dados é um **chef de cozinha**. Ele não começa decorando o prato (`SELECT`); ele começa pegando os ingredientes no estoque.

1º FROM e JOIN (A Despensa)

Antes de qualquer coisa, o SQL precisa saber: **de onde vêm os dados?**

Ele vai até as tabelas (ex: `instrutor` e `ministra`) e as une. Conceitualmente, ele cria um grande mesclado de dados (o produto cartesiano filtrado pelas chaves PK/FK). Se não houver essa conexão inicial, não há dados para trabalhar.

2º WHERE (A Triagem)

Agora que o "mesclado" de tabelas foi criado, o SQL olha linha por linha e joga fora o que não presta.

- **Exemplo:** "Só quero professores do departamento de Física".
- O `WHERE` reduz o tamanho do conjunto de dados logo no início para economizar memória.

3º GROUP BY (A Organização)

Com as linhas que sobraram, o SQL começa a montar "pacotes" ou "balaio".

- **Exemplo:** "Separe esses professores em pilhas por departamento".

- As linhas individuais deixam de existir sozinhas e passam a fazer parte de um grupo.

4º HAVING (O Controle de Qualidade dos Grupos)

Aqui é onde muitos se confundem. O **HAVING** é um filtro que só funciona para os **pacotes** que acabamos de montar.

- **Exemplo:** "Destaque apenas os pacotes (departamentos) onde a soma dos salários é maior que 100 mil".
- Se o pacote não atingir a meta, ele é descartado inteiro.

5º SELECT (A Apresentação do Prato)

Só agora, no final, é que o SQL olha para o que você pediu para "mostrar".

- Ele calcula as médias e somas finais.
- Ele escolhe quais colunas vão aparecer.
- Ele coloca os apelidos (**Aliases**) que você definiu (ex: **AS total_gasto**).

Curiosidade: É por isso que você não pode usar um apelido criado no **SELECT** dentro do **WHERE**. Quando o banco passou pelo **WHERE**, o **SELECT** (e o apelido que você deu) ainda nem tinha sido processado.

Resumo Visual da Aula

RESUMO VISUAL DA AULA

A VIAGEM DO DADO NO SQL ↴



Com essa visão clara da 'ordem de **trás para frente**', você não apenas escreve **queries**, mas entende como o banco de dados '**pensa**'. **Parabéns por concluir todos os módulos desta jornada SQL!**

